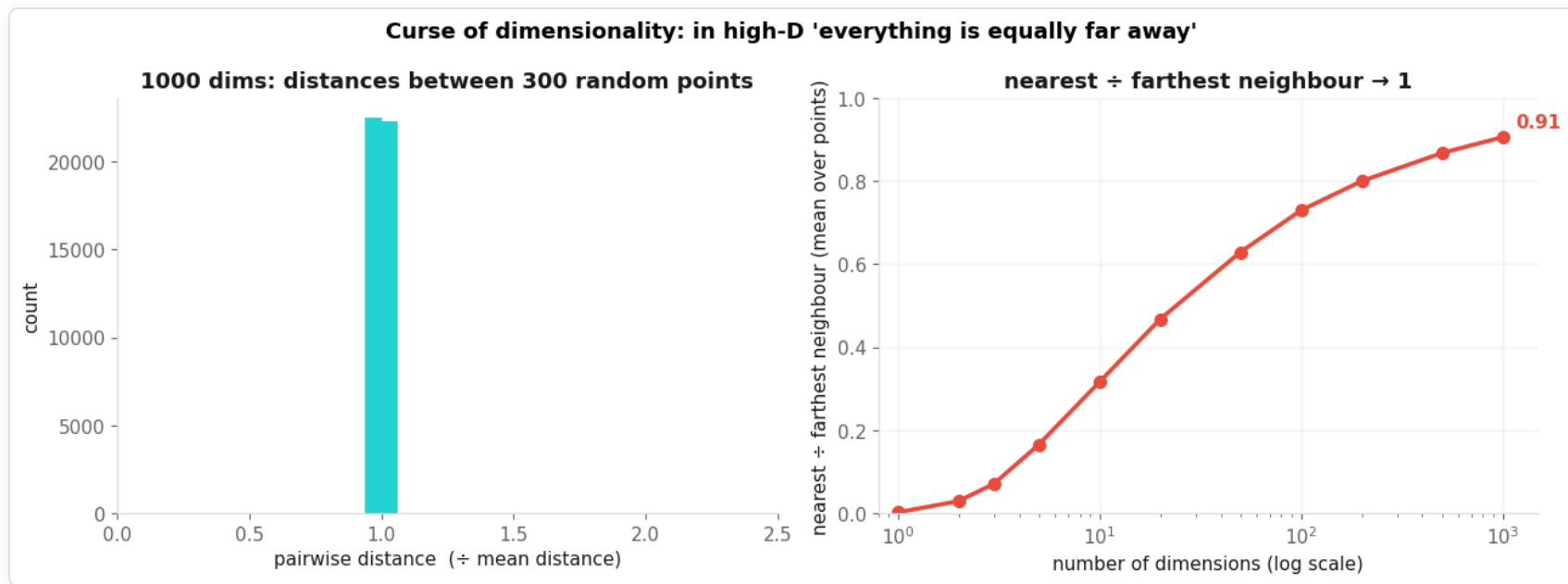


Dimensionality Reduction

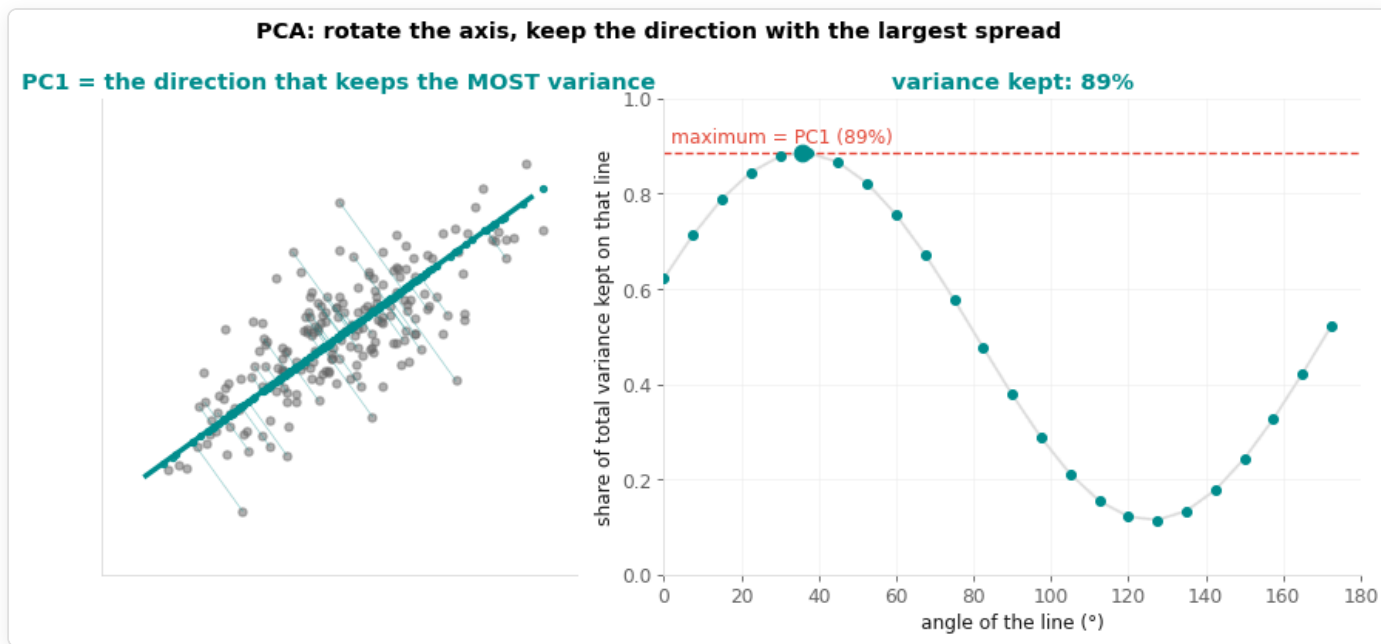
Applied Machine Learning — Session 3, Chapter 3

The Curse of Dimensionality



As dimensions grow, random points become **equally far from each other** $\rightarrow$ distance-based methods (KNN, K-Means, DBSCAN) lose their signal; more data is needed; overfitting risk grows; plotting is impossible beyond 3-D.

PCA: Find the Direction With the Most Spread



PC1 = direction of maximum variance • **PC2** = max remaining variance, $\perp$ PC1 • ... keep the top ones, drop the rest.

PCA in Code — Always in a Pipeline With Scaling

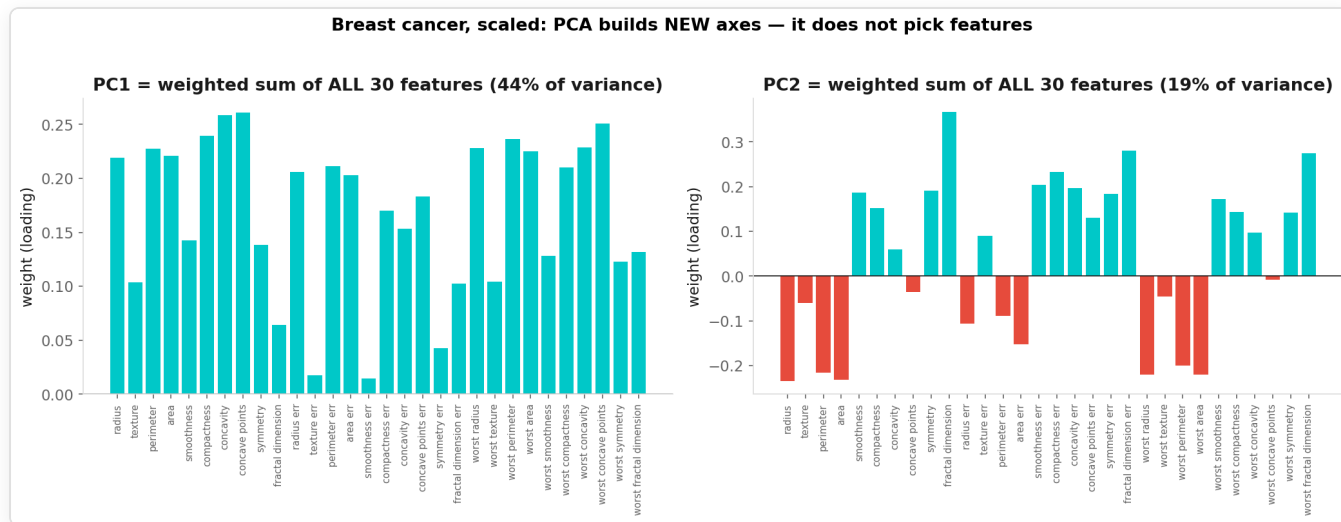
```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

pca_pipe = make_pipeline(StandardScaler(), PCA(n_components=2)) # scale first!
X_2d = pca_pipe.fit_transform(X)                               # no y – unsupervised

pca = pca_pipe['pca']
pca.explained_variance_ratio_      # e.g. [0.44, 0.19] → share of variance per component
pca.explained_variance_ratio_.sum() # how much of the total spread the 2-D picture keeps
pca.components_                   # the directions: one row of weights per PC (next slide)
```

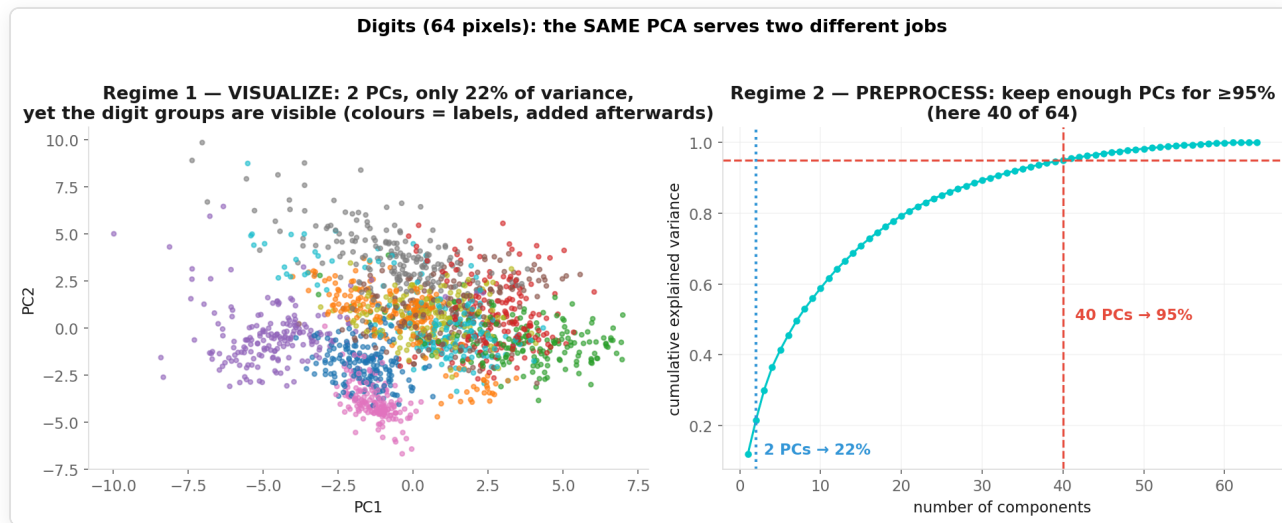
Without scaling, the feature with the biggest numbers *is* PC1 (same lesson as K-Means).

PCA ≠ Feature Selection



Each PC is a **weighted sum of ALL original features** (`components_`). PCA does not pick "the best 2 columns" — it builds 2 new axes. Consequence: PCs are harder to interpret; if you need "which raw features matter", use feature selection / model importances instead.

How Many Components? — Two Different Jobs



Visualise: 2 PCs — the % is whatever it is (22 % here); judge the picture, not the number.

Preprocess: keep enough PCs for $\approx 90\text{--}95\%$ → `PCA(n_components=0.95)` (float = variance share; here 40 of 64).

PCA as Preprocessing (Inside the Pipeline!)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

pipe = make_pipeline(StandardScaler(), PCA(n_components=0.95),
                    RandomForestClassifier(random_state=42))
scores = cross_val_score(pipe, X, y, cv=5) # scaler+PCA re-fitted on each training fold
pipe.fit(X_train, y_train); pipe['pca'].n_components_ # how many PCs were kept
```

-  PCA is fitted **only on the training part** of each fold → no leakage (same rule as imputation/scaling in Ch02)
- Helps when features are many & correlated, or for speed/denoising; a tree model on 30 clean features often does **not** improve – always compare with the no-PCA pipeline and a **DummyClassifier baseline**

t-SNE: Non-Linear Visualisation

Digits: same 64-D data, two 2-D pictures (colours = true labels, added afterwards)

PCA — linear projection, axes = variance, distances meaningful

t-SNE — non-linear, keeps NEIGHBOURS; distances between blobs are NOT meaningful



```
from sklearn.manifold import TSNE
X_tsne = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X_scaled)
```


t-SNE: What You Can and Cannot Conclude

- ✓ "These samples sit together → they are similar in the original space"
- ✓ "This island looks isolated → maybe a distinct group — verify with clustering/domain knowledge"
- ✗ "Cluster A is 3× farther from B than from C" — inter-cluster distances are **not** meaningful
- ✗ "The plot shows 5 clusters → the data has 5 groups" — perplexity and seed change the picture
- ✗ Using t-SNE coordinates as model features: **there is no** `.transform()` for new data → cannot go into a train/test pipeline

Different `random_state` → different picture. **Visualisation only.** (UMAP: similar idea, faster, has `transform`, `pip install umap-learn`.)

Quick Check

Q1. Your 2-D PCA of a 500-feature dataset shows 18 % explained variance, but the two patient groups separate nicely. Good or bad?

Q2. A colleague fits PCA on the full dataset, then splits train/test and reports 97 % accuracy. What is the problem?

Quick Check

Q1. Your 2-D PCA of a 500-feature dataset shows 18 % explained variance, but the two patient groups separate nicely. Good or bad?

→ Fine for a picture: 18 % is normal for 500 features; judge the plot by the structure you see. For *preprocessing* you would keep many more PCs ($\approx 90-95$ %).

Q2. A colleague fits PCA on the full dataset, then splits train/test and reports 97 % accuracy. What is the problem?

Quick Check

Q1. Your 2-D PCA of a 500-feature dataset shows 18 % explained variance, but the two patient groups separate nicely. Good or bad?

→ Fine for a picture: 18 % is normal for 500 features; judge the plot by the structure you see. For *preprocessing* you would keep many more PCs ($\approx 90-95$ %).

Q2. A colleague fits PCA on the full dataset, then splits train/test and reports 97 % accuracy. What is the problem?

→ Leakage: the components were computed using test rows. Put PCA in the Pipeline so it is fitted per training fold. (Often a small effect for PCA — but the same rule as scaling/imputation.)

Live Demo

→ Open `02-examples/ch09_dimensionality_reduction_examples.ipynb` (~5 min)

1. Digits: scree plot → how many PCs for 80/90/95 %?
2. 2-D PCA vs t-SNE side by side
3. PCA inside a classification pipeline (with baseline) — does it help?

Now: Exercises!

→ Open `03-exercises/ch09_dimensionality_reduction_exercises.ipynb`

Task (breast cancer, 30 features): scree plot → 2-D picture → PCA inside a classifier pipeline vs. no-PCA vs. dummy baseline.

Bonus: t-SNE comparison • read the PC1 loadings

~10 minutes

Key Takeaways

- High-D → distances lose meaning → reduce dimensions
- PCA: **new axes** = weighted combinations of all features, ordered by variance; scale first; unsupervised
- 2 PCs for pictures (whatever %); ≈ 95 % for preprocessing — inside the Pipeline (no leakage)
- PCA $\neq$ feature selection
- t-SNE: neighbours only, no distances, no `.transform()` → visualisation only

Next: Session 4

Reinforcement Learning

"What if the machine learns by trial and error – like us?"